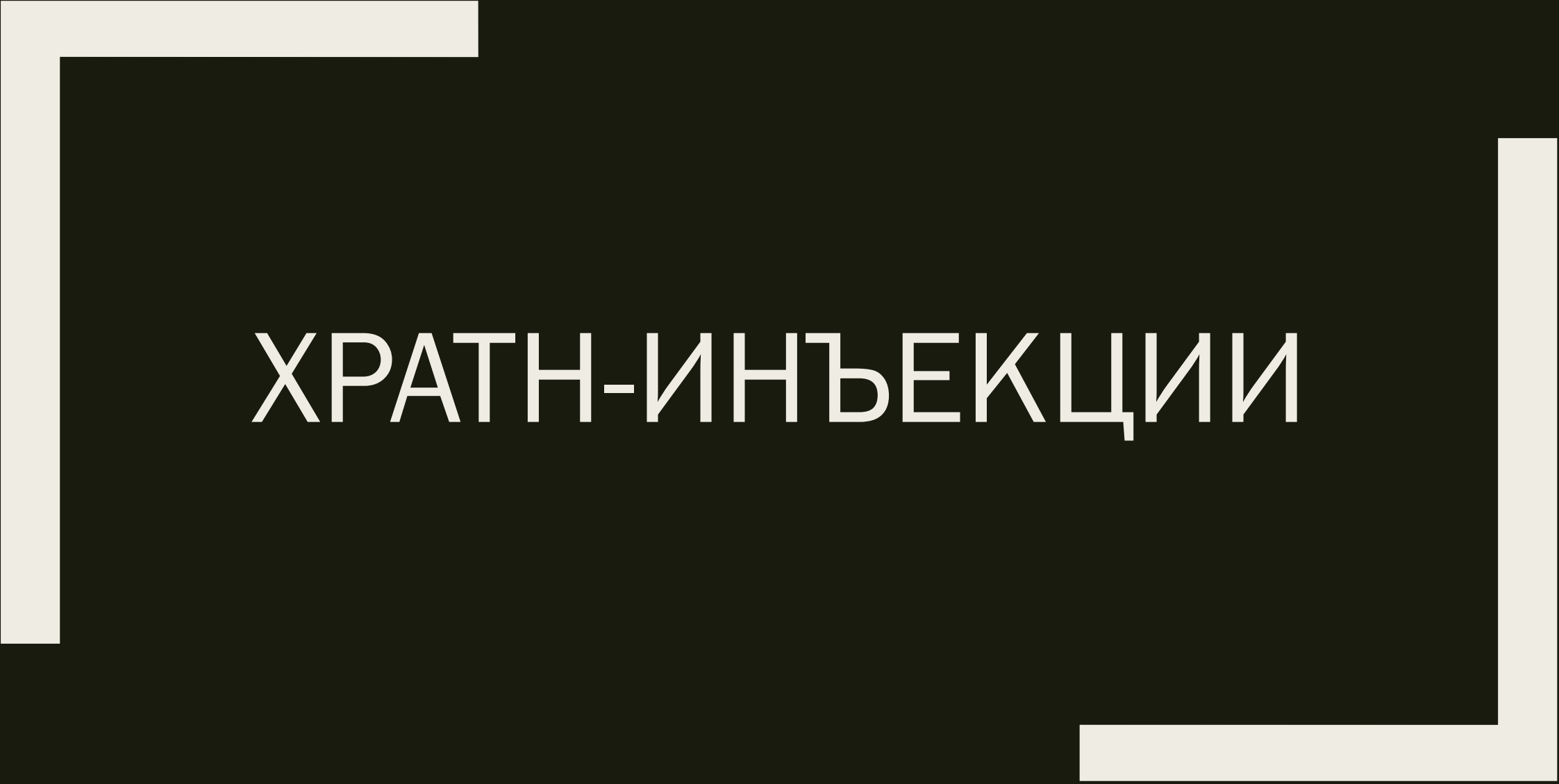




ХРАТН-ИНЪЕКЦИИ

XPath

XPath - язык запросов для XML документов, в общем похожий на SQL для баз данных. Правда вместо таблиц, колонок и строчек XPath оперирует нодами в XML дереве. Но подобно SQL, XPath может быть уязвим для инъекции в случае если введенные данные недостаточно проверяются на стороне сервера.

В чем опасность XPath инъекций?

XPath 1.0 - стандартный язык в отличие от SQL, который имеет множество «диалектов», основанных на относительно слабом синтаксисе.

XPath 1.0 позволяет получить все объекты базы (XML объекты). В SQL во многих случаях мы не можем простым SELECT добраться до всех объектов базы данных.

XPath 1.0 не имеет разграничений прав для доступа к базе данных в то время как в SQL некоторые части БД могут быть недоступны из-за недостатка прав.

Пример 1

Представим что у нас есть XML база и аутентификация, основанная на ней:

```
<?xml version='1.0' encoding='ISO-8859-1'?> <users>
```

```
  <user>
```

```
    <id> 1 </id>
```

```
    <username> admin </username>
```

```
    <password> xp8th! </password>
```

```
  </user>
```

```
  <user>
```

```
    <id> 2 </id>
```

```
    <username> test </username>
```

```
    <password> test987 </password>
```

```
  </user>
```

```
  <user>
```

```
    <id> 3 </id>
```

```
    <username> bigolnerd </username>
```

```
    <password> nerdsneedlovetoo</password>
```

```
  </user>
```

```
</users>
```

Пример 1

Код, реализующий аутентификацию:

```
String username = req.getParameter("username");
```

```
String password = req.getParameter("password");
```

```
XPathFactory factory = XPathFactory.newInstance();
```

```
Xpath xpath = factory.newXPath();
```

```
File file = new File("/usr/webappdata/users.xml");
```

```
InputSource src = new InputSource(new FileInputStream(file));
```

```
XPathExpression expr = xpath.compile("//users[username/text()=' " +  
username + " ' and password/text()=' " + password + " ']/id/text()");
```

```
String id = expr.evaluate(src);
```

Пример 1

Этот код загружает XML документ и запросом получает из него ID, привязанный к введенному пользователем логину и паролю. Предположим это "admin" и "xp8th!". В таком случае запрос будет такой:

```
//users[username/text()='admin' and password/text()='xp8th!']/id/text()
```

Никаких проверок не производится и можно применить давно знакомый нам подход использовав ' или '1'='1;

```
//users[username/text()='admin' and password/text()=' or '1'='1']/id/text()
```

Запрос вернет ID для пользователя admin с пустым паролем или при условии 1=1, что всегда истина.

Пример 2

Представим, что у нас есть такой документ:

```
<?xml version="1.0" encoding="utf-8" ?>
<orders>
  <customer id="1">
    <name>Bob Smith</name>
    <email>bob.smith@bobsmithinc.com</email>
    <creditcard>1234567812345678</creditcard>
    <order>
      <item>
        <quantity>1</quantity>
        <price>10.00</price>
        <name>Sprocket</name>
      </item>
      <item>
        <quantity>2</quantity>
        <price>9.00</price>
        <name>Cog</name>
      </item>
    </order>
  </customer>
  ...
</orders>
```

Пример 2

Сайт позволяет пользователю осуществлять поиск по своим предыдущим заказам по цене. XPath запрос в приложении выглядит примерно так:

```
string query = "/orders/customer[@id='" + customerId +  
"/]/order/item[price >= '" + priceFilter + '']";
```

Если оба поля `customerId` и `priceFilter` не проверяются на ввод, атакующий может использовать инъекцию. Введя следующие значения нападающий получит весь XML документ:

```
'] | /* | /foo[bar='
```

Запрос будет выглядеть так:

```
string query = "/orders/customer[@id=''] | /* | /foo[bar='']/order/item[price >= '" + priceFilter + '']";
```

Одним простым запросом мы получаем всю базу данных.

Зачем использовать XML вместо баз данных?

Многие XML приложения используют XML дампы баз данных. Идея состоит в том, что можно ВСЕ ЧТО УГОДНО поместить в XML и потом использовать приложение или некоторый код для парсинга тех данных, которые вам нужны (В частности, например, на Хакере XML базы применяются для разгрузки занятого SQL сервера. Сформировав дампы мы перенесли проблему получения нужных данных со сложных и тяжелых SQL запросов на файловую систему и простое чтение XML файлов.) Проблема же в том, что нет никакого контроля над уровнем доступа и если ваше приложение или код читает XML документ, то существует возможность того, что ЛЮБЫЕ данные в нем могут быть просмотрены.

Зачем использовать XML вместо баз данных?

Если ваш сайт использует XML документы для хранения данных и пользовательский ввод используется для построения запросов, то вполне возможно, что он уязвим для XPath инъекции.

Защита от XPath инъекций

Лучший путь это, конечно, прямые руки разработчиков - проверка всех полученных от пользователя данных на неправильные символы и слова. Наиболее оптимально создать список разрешенных символов, например, для ввода номера кредитной карты это будут цифры от 0 до 9, для, например, имен - только буквы. Второй способ - параметризация запросов. Вместо того, чтобы собирать строку запроса в приложении на лету, динамически, целесообразнее создать прекомпилированный запрос и уже передавать переменные не выражениями, а параметрами.